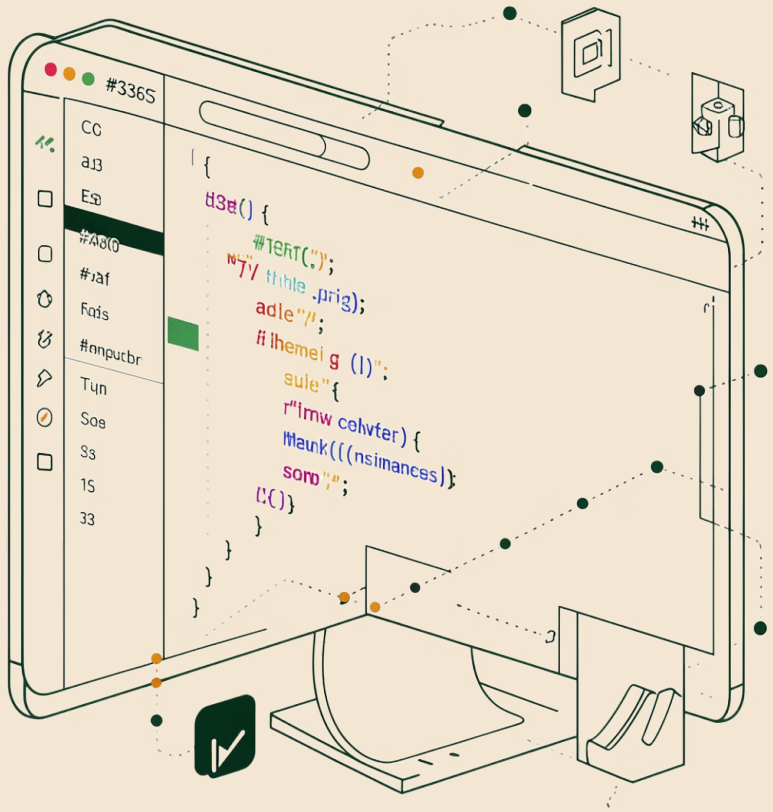
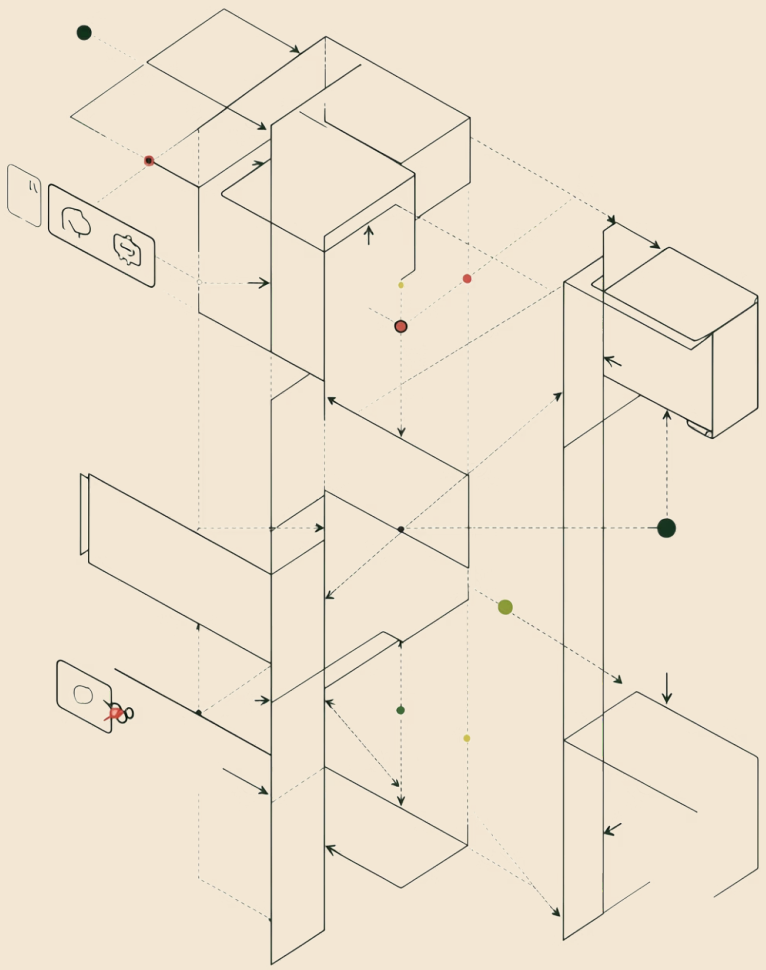


CSS Display Property: All Values Explained

A classroom guide to every display value — from block and inline to Flexbox and Grid.



What Does `display` Do?

The `display` property controls how an element is rendered in the layout — whether it starts on a new line, how much space it takes, and how its children are arranged.

Every HTML element has a default display value. Changing it is one of the most powerful tools in CSS.

Layout Flow

Controls new line vs. inline placement

Sizing

Determines how width & height apply

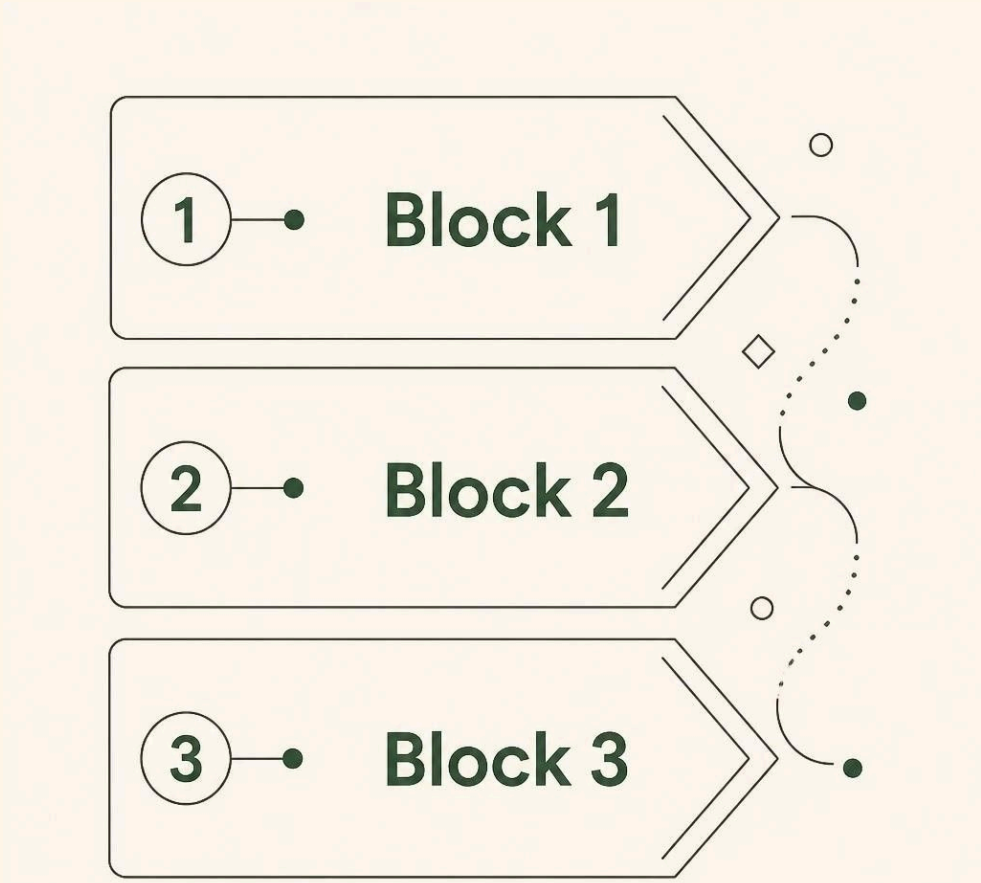
Children

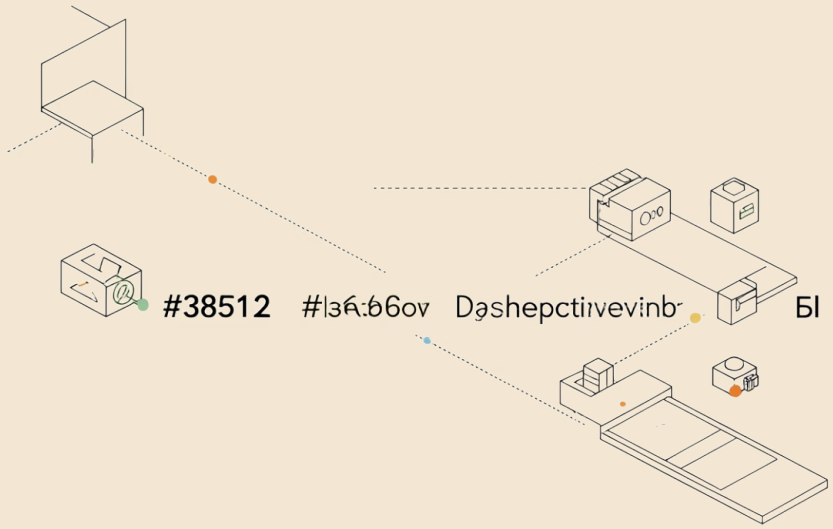
Sets how child elements are arranged

display: block

Block elements start on a **new line** and stretch to fill the full available width. They fully respect `width`, `height`, `margin`, and `padding` on all sides.

> **Default for:** `div`, `p`, `h1`–`h6`, `section`, `article`, `header`, `footer`, `nav`





display: inline

Flows Within Text

Does **not** start on a new line — sits alongside surrounding content in the text flow.

No Width or Height

`width` and `height` have **no effect**. Size is determined by content only.

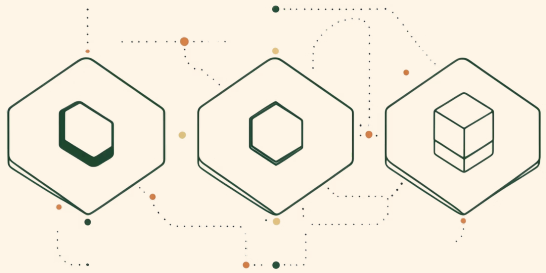
Limited Spacing

Top/bottom `margin` and `padding` are ignored for layout purposes.

Default For

`span`, `a`, `strong`, `em`, `label`

display: inline-block



Sits **beside other elements** like inline, but accepts `width`, `height`, and full `margin`/`padding` like block — the best of both worlds.

- Commonly used for **buttons, nav links, and badges**
- **Key difference from inline:** top/bottom spacing is fully respected
- Three boxes can sit side by side, each with explicit dimensions

display: none vs visibility: hidden

display: none

Element is **completely removed** from the layout. No space is reserved — other elements fill the gap.

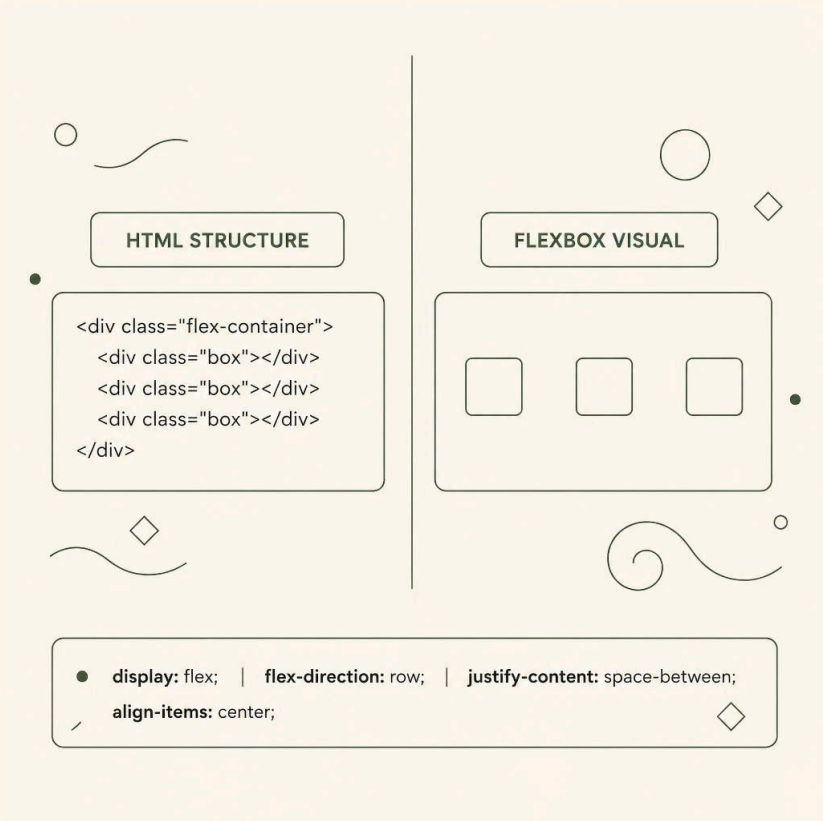
✅ Use for: toggling menus, modals, conditional UI

visibility: hidden

Element is **invisible** but its space is still reserved in the layout. A blank gap remains.

⚠️ Use when: you need to preserve layout spacing

display: flex — Flexbox Basics



Makes the container a **flex container**; direct children become flex items. Default direction is **row** (left to right).

Perfect for: nav bars, button groups, centering content, card rows

justify-content

Aligns items on the main axis

align-items

Aligns items on the cross axis

gap / flex-wrap

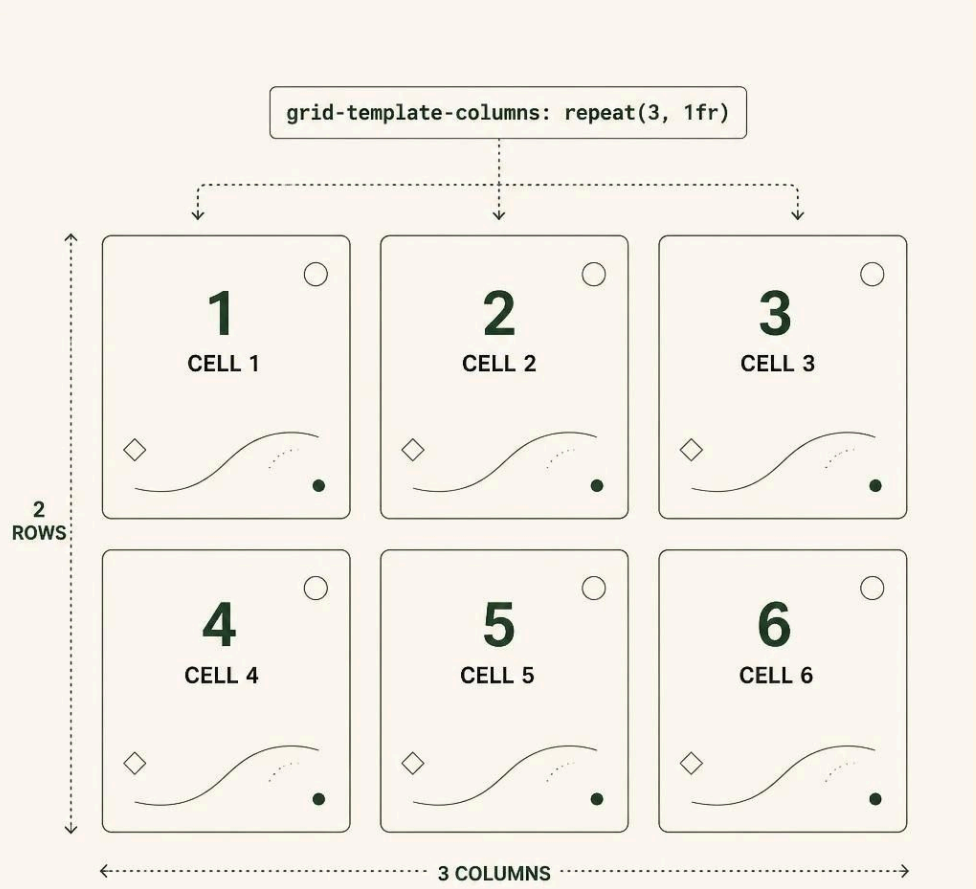
Spacing and wrapping control

`display: grid` — Grid Basics

Makes the container a **grid container**; children are placed into rows and columns. Define structure with `grid-template-columns` and `grid-template-rows`.

- `grid-template-columns: repeat(3, 1fr)` — three equal columns
- `gap` — spacing between cells
- `grid-column` / `grid-row` — span multiple cells

Perfect for: page layouts, image galleries, dashboards, card grids



Flex vs Grid: When to Use Which

	Flexbox	Grid
Dimension	One-dimensional (row *or* column)	Two-dimensional (rows *and* columns)
Best for	Nav bars, button groups, centering	Page layouts, galleries, dashboards
Control	Content drives size	Layout drives size
Typical use	`display: flex` + `justify-content`	`display: grid` + `grid-template-columns`

display: inline-flex & inline-grid

inline-flex

A flex container that behaves like an **inline element** — sits in text flow but arranges children with flexbox rules.

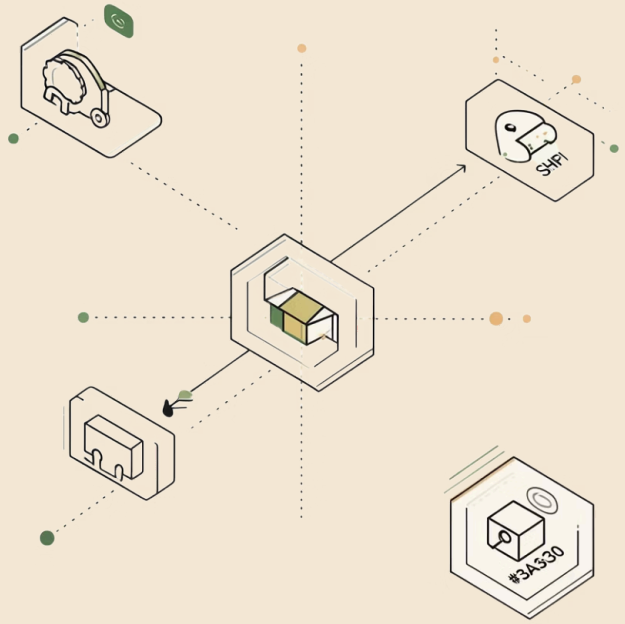
e.g. icon + label combos, pill badges

inline-grid

Same idea — a grid container that **doesn't break onto its own line**. Useful for small self-contained grid components.

e.g. tag chips, mini data grids

Both are niche but powerful — use them when you need a flex or grid layout that must live *within* a line of text or alongside other inline content.



Quick Reference: All `display` Values

Value	Behaviour	Common Use
`block`	Full width, starts on a new line	`div`, `p`, headings
`inline`	Content width, no new line	`span`, `a`, `strong`
`inline-block`	Inline position + block sizing	Buttons, nav links
`none`	Removed from layout entirely	Toggles, modals
`flex`	1D flexible layout (row or column)	Nav bars, card rows
`inline-flex`	Inline flex container	Icon + label combos
`grid`	2D row + column layout	Page layouts, galleries
`inline-grid`	Inline grid container	Small grid components